Timer/WatchDog /RTC

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



Adapted from

- ***Richard Kuo***
- ***Assistant Professor***

1

Outline

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

- **NuMicro MCU Timer/WatchDog Timer/Real-Time Clock**
  - **6.NuMicro_Timer_WatchDog_RTC.ppt**
- **Exercise: Timer – one-shot、periodic、continuous**
  - **Smpl_Timer_LED**
  - **Smpl_Timer_Counter**
  - **Smpl_Timer_LCD**
- **Exercise: Watch-Dog Timer (Smpl_WDT)**
- **Exercise: Real-Time Clock (Smpl_RTC)**
- **Exercise: using Timer + WatchDog Timer + RTC alarm (Smpl_Timer_WDT_RTC)**
- **Exercise: using Timer Capture pin (Smpl_Timer_SR04)**

3

Timer Features

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

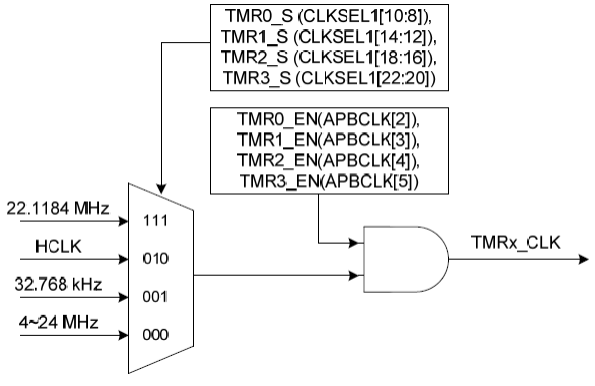
- Four 24-bit up-counting timer
- A prescaler (divider) for each Timer
- Selectable 5 clock source
- Four modes : one shot, periodic, toggle, continuous
- Timeout Period calculation

Time out period = (Period of timer clock input) * (8-bit Prescale + 1) * (24-bit TCMP)

4

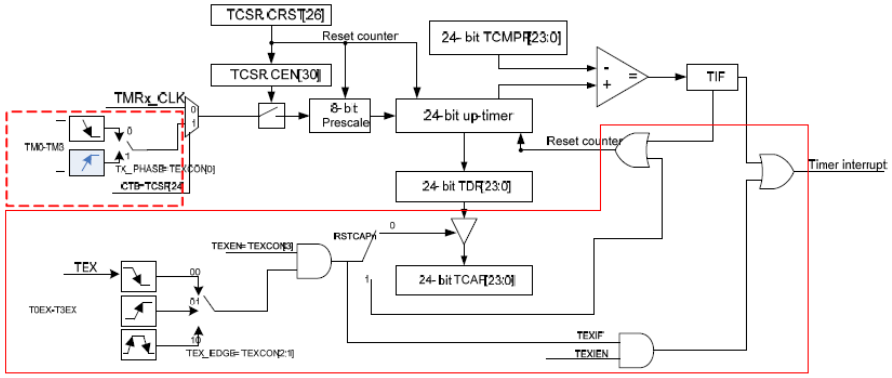
Timer Clock Source

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



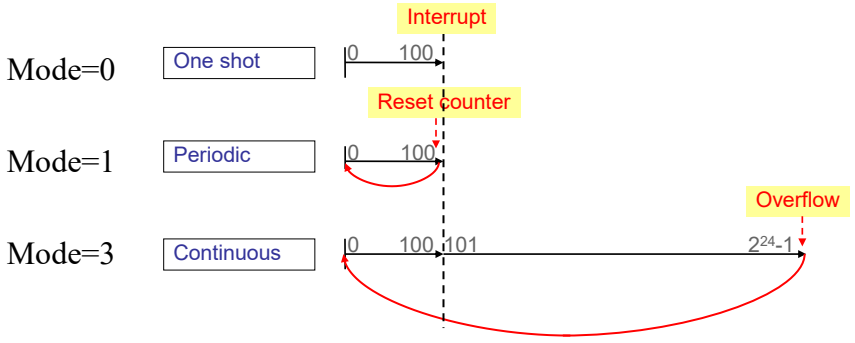
5

Timer Block Diagram



Timer operation modes

- Ex. TCMPR = 100



Smpl_Timer_LED


```
//
// Smpl_Timer_LED : on/off LED every second
//
#include <stdio.h>
#include "NUC1xx.h"
#include "SYS.h"
#include "GPIO.h"

#define  ONESHOT  0   // counting and interrupt when reach TCMPR number, then stop
#define  PERIODIC 1   // counting and interrupt when reach TCMPR number, then counting from 0
      again
#define  TOGGLE   2   // keep counting and interrupt when reach TCMPR number, tout toggled
      (between 0 and 1)
#define  CONTINUOUS 3 // keep counting and interrupt when reach TCMPR number
```

8

Smpl_Timer_LED


```
volatile uint32_t ledState = 0;

void TMR0_IRQHandler()
{
     ledState = ~ ledState;    // changing ON/OFF state
     if(ledState) DrvGPIO_ClrBit(E_GPC,12);
     else         DrvGPIO_SetBit(E_GPC,12);
     TIMER0->TISR.TIF = 1;  // clear Interrupt flag
}
```

9

Smpl_Timer_LED


```
void InitTIMER0(void)
{
    /* Step 1. Enable and Select Timer clock source */
    SYSCLK->CLKSEL1.TMR0_S = 0;          //Select 12Mhz for Timer0 clock source
    SYSCLK->APBCLK.TMR0_EN = 1;          //Enable Timer0 clock source
    /* Step 2. Select Operation mode */
    TIMER0->TCSR.MODE = PERIODIC; //Select operation mode
    /* Step 3. Select Time out period = (Period of timer clock input) * (8-bit Prescale + 1) * (24-bit TCMP)*/
    TIMER0->TCSR.PRESCALE = 255;  // Set Prescale [0~255]
    TIMER0->TCMPR = 46875;       // Set TCMPR [0~16777215]
    //Timeout period = (1 / 12MHz) * ( 255 + 1 ) * 46875 = 1 sec
    /* Step 4. Enable interrupt */
    TIMER0->TCSR.IE = 1;
    TIMER0->TISR.TIF = 1;          //Write 1 to clear for safty
    NVIC_EnableIRQ(TMR0_IRQn);              //Enable Timer0 Interrupt
    /* Step 5. Enable Timer module */
    TIMER0->TCSR.CRST = 1;    //Reset up counter
    TIMER0->TCSR.CEN = 1;     //Enable Timer0
//  TIMER0->TCSR.TDR_EN=1;  // Enable TDR function
}
```


Smpl_Timer_LED


```
int32_t main(void)
{
    UNLOCKREG();
    SYSCLK->PWRCON.XTL12M_EN = 1;//Enable 12MHz Crystal
    DrvSYS_Delay(5000); // waiting for 12MHz crystal stable
    SYSCLK->CLKSEL0.HCLK_S = 0;
    LOCKREG();

    DrvGPIO_Open(E_GPC,12, E_IO_OUTPUT); // set GPC12 output for LED
    InitTIMER0();   // Set Timer Ticking

    while(1)
    { // Do Nothing
    }
}
```

Smpl_Timer_LCD

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



12

Smpl_Timer_LCD

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

```
//
// Smpl_Timer_LCD
//
// Timer0 set at one-shot   @ 1 sec or 1 Hz
// Timer1 set at periodic   @ 1 sec or 1 Hz
// Timer2 set at contiuous  @ 1 sec or 1 Hz

#include <stdio.h>
#include <string.h>
#include "NUC1xx.h"
#include "LCD.h"

#define  ONESHOT  0   // counting and interrupt when reach TCMPR number, then stop
#define  PERIODIC 1   // counting and interrupt when reach TCMPR number, then counting from 0
      again
#define  TOGGLE   2   // keep counting and interrupt when reach TCMPR number, tout toggled
      (between 0 and 1)
#define  CONTINUOUS 3 // keep counting and interrupt when reach TCMPR number

static uint16_t Timer0Counter=0;
static uint16_t Timer1Counter=0;
static uint16_t Timer2Counter=0;
```

13

InitTimer0


```
void InitTIMER0(void)
{
    /* Step 1. Enable and Select Timer clock source */
    SYSCLK->CLKSEL1.TMR0_S = 0;          //Select 12Mhz for Timer0 clock source
    SYSCLK->APBCLK.TMR0_EN = 1;          //Enable Timer0 clock source
    /* Step 2. Select Operation mode */
    TIMER0->TCSR.MODE=ONESHOT;           //Select once mode for operation mode
    /* Step 3. Select Time out period= (Period of timer clock input) * (8-bit Prescale + 1) * (24-bit TCMP)*/
    TIMER0->TCSR.PRESCALE=255;           // Set Prescale [0~255]
    TIMER0->TCMPR = 46875;               // Set TCMPR [0~16777215]
                                         // (1/12000000)*(255+1)* 46875 = 1 sec / 1 Hz
    /* Step 4. Enable interrupt */
    TIMER0->TCSR.IE = 1;
    TIMER0->TISR.TIF = 1;                //Write 1 to clear for safty
    NVIC_EnableIRQ(TMR0_IRQn);           //Enable Timer0 Interrupt
    /* Step 5. Enable Timer module */
    TIMER0->TCSR.CRST = 1;               //Reset up counter
    TIMER0->TCSR.CEN = 1;                //Enable Timer0
//  TIMER0->TCSR.TDR_EN=1;               // Enable TDR function
}
```


IRQHandler


```
void TMR0_IRQHandler(void) // Timer0 interrupt subroutine
{
    char TEXT1[16]="Timer0:       ";
    Timer0Counter+=1;
    sprintf(TEXT1+7,"%d",Timer0Counter);
    print_Line(1, TEXT1);
    TIMER0->TISR.TIF =1;
}

void TMR1_IRQHandler(void) // Timer1 interrupt subroutine
{
    char TEXT2[16]="Timer1:       ";
    Timer1Counter+=1;
    sprintf(TEXT2+7,"%d",Timer1Counter);
    print_Line(2, TEXT2);
    TIMER1->TISR.TIF =1;
}
```

Smpl_Timer_LCD


```
int32_t main (void)
{
    UNLOCKREG();
    SYSCLK->PWRCON.XTL12M_EN = 1;//Enable 12MHz Crystal
    SYSCLK->CLKSEL0.HCLK_S = 0;
    LOCKREG();

    init_LCD();
    clear_LCD();

    print_Line(0,"Smpl_Timer");
    InitTIMER0();
    InitTIMER1();
    InitTIMER2();

    while(1)
    {
        __NOP(); // No Operation
    }
}
```

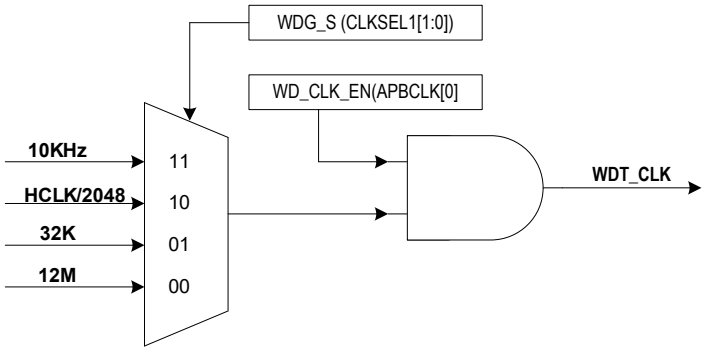
16

WatchDog Timer

- 18-bit up counter
- 8 selections of time out period
- Support function:
  - WDT interrupt
  - WDT reset
  - WDT wakeup

17

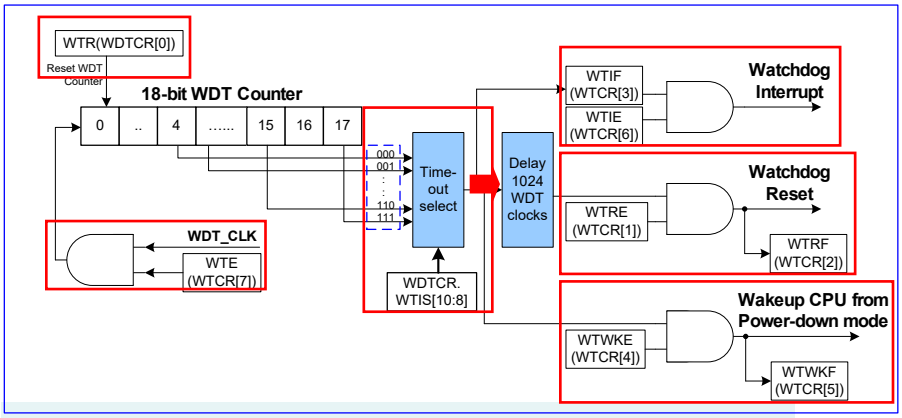
WDT Clock = Generato



18

Watch mode Block Diagram

- WDT can cause a WDT Interrupt or WDT Reset to CPU after a fixed delay period.
- WDT also can wakeup CPU form Power-down mode



19

WDT Time out selection

WTIS	Interrupt Timeout	Watchdog Reset Timeout	WTR Timeout Interval for Reset (WDT_CLK=12MHz)	WTR Timeout Interval for Reset (WDT_CLK=32KHz)
000	2^4 WDT_CLK (16 WDT_CLK)	(2^4 + 1024) WDT_CLK (1040 WDT_CLK)	86.67us	32.5 ms
001	2^6 WDT_CLK	(2^6 + 1024) WDT_CLK	90.67 us	34 ms
010	2^8 WDT_CLK	(2^8 + 1024) WDT_CLK	106.67 us	40 ms
011	2^{10} WDT_CLK	(2^{10} + 1024) WDT_CLK	170.67 us	64 ms
100	2^{12} WDT_CLK	(2^{12} + 1024) WDT_CLK	426.67 us	160 ms
101	2^{14} WDT_CLK	(2^{14} + 1024) WDT_CLK	1.45 ms	544 ms
110	2^{16} WDT_CLK	(2^{16} + 1024) WDT_CLK	5.55 ms	2080 ms
111	2^{18} WDT_CLK	(2^{18} + 1024) WDT_CLK	21.93 ms	8224 ms

20

Smpl_WDT



21

InitWDT()


```
// Initialize Watch-Dog Timer
void InitWDT(void)
{
    UNLOCKREG();
    /* Step 1. Enable and Select WDT clock source */
    SYSCLK->CLKSEL1.WDT_S =3;//Select 10Khz for WDT clock source
    SYSCLK->APBCLK.WDT_EN =1;//Enable WDT clock source

    /* Step 2. Select Timeout Interval */
    WDT->WTCR.WTIS=5;
    /* Step 3. Disable Watchdog Timer Reset function */
    WDT->WTCR.WTRE = 0;
    /* Step 4. Enable WDT interrupt */
    WDT->WTCR.WTIF =1;//Write 1 to clear for safety
    WDT->WTCR.WTIE = 1;
    NVIC_EnableIRQ(WDT_IRQn);
    /* Step 5. Enable WDT module */
    //Enable WDT
    WDT->WTCR.WTE = 1;
     //Clear WDT counter
    WDT->WTCR.WTR = 1;
    LOCKREG();
}
```


WDT_IRQHandler()


```
// Watch-Dog Timer Interrupt Handler
void WDT_IRQHandler(void)
{
    UNLOCKREG();
    WDT->WTCR.WTIF =1;
    WDT->WTCR.WTR = 1;
    UNLOCKREG();
    print_Line(1, "WatchDog Timer  ");
    print_Line(2, "Interrupt !!!   ");
}
```

Smpl_WDT


```
int32_t main (void)
{
    UNLOCKREG();
    SYSCLK->PWRCON.XTL32K_EN = 1;//Enable 32KHz for RTC clock source
    SYSCLK->PWRCON.XTL12M_EN = 1;//Enable 12MHz crystal
    SYSCLK->CLKSEL0.HCLK_S = 0;
    LOCKREG();

    init_LCD();
    clear_LCD();

    print_Line(0,"Smpl_WDT");

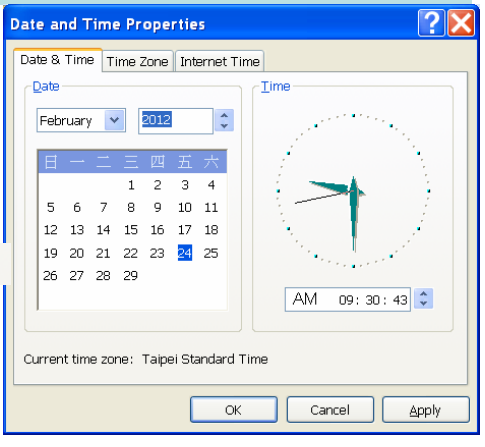
    InitWDT();

    while(1)
    {
        __NOP();
    }
}
```


RTC Features

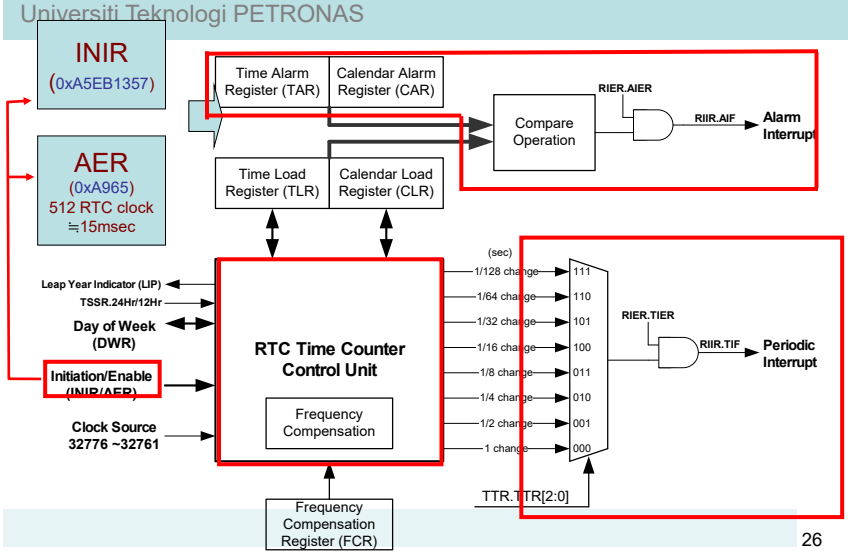
- RTC counter
  - Time counter
  - Calendar counter
  - Day of week
  - Leap year
  - BCD format
    - TLR=0x00093043

      AM 09: 30: 43
    - CLR=0x00120229
- A set of Alarm
- Alarm and time tick interrupt
- Wakeup function
- 32KHz compensation



RTC Block Diagram

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



RTC Access

Department of Electrical and Electronic Engineering,

Register	Offset	R/W	Description	Reset Value	AER[16]=0
RTC_BA = 0x4000_8000					
INIR	RTC_BA+0x000	R/W	RTC Initiation Register(A5EB1357)	0x0000_0000	R/W
AER	RTC_BA+0x004	R/W	RTC Access Enable Register(A965)	0x0000_0000	R/W
FCR	RTC_BA+0x008	R/W	RTC Frequency Compensation Register	0x0000_0700	-
TLR	RTC_BA+0x00C	R/W	Time Loading Register	0x0000_0000	R
CLR	RTC_BA+0x010	R/W	Calendar Loading Register	0x0005_0101	R
TSSR	RTC_BA+0x014	R/W	Time Scale Selection Register	0x0000_0001	R/W
DWR	RTC_BA+0x018	R/W	Day of the Week Register	0x0000_0006	R
TAR	RTC_BA+0x01C	R/W	Time Alarm Register	0x0000_0000	-
CAR	RTC_BA+0x020	R/W	Calendar Alarm Register	0x0000_0000	-
LIR	RTC_BA+0x024	R	Leap year Indicator Register	0x0000_0000	R
RIER	RTC_BA+0x028	R/W	RTC Interrupt Enable Register	0x0000_0000	R/W
RIIR	RTC_BA+0x02C	R/C	RTC Interrupt Indicator Register	0x0000_0000	R/C
TTR	RTC_BA+0x030	R/W	RTC Time Tick Register	0x0000_0000	-

Access enable

Access disable

24-hour/12-hour Mapping Table

BCD Format

24-hour time scale	12-hour time scale	24-hour time scale	12-hour time scale (PM time + 0x20)
0x00	0x12(AM12)	0x12	0x32(PM12)
0x01	0x01(AM01)	0x13	0x21(PM01)
0x02	0x02(AM02)	0x14	0x22(PM02)
0x03	0x03(AM03)	0x15	0x23(PM03)
0x04	0x04(AM04)	0x16	0x24(PM04)
0x05	0x05(AM05)	0x17	0x25(PM05)
0x06	0x06(AM06)	0x18	0x26(PM06)
0x07	0x07(AM07)	0x19	0x27(PM07)
0x08	0x08(AM08)	0x20	0x28(PM08)
0x09	0x09(AM09)	0x21	0x29(PM09)
0x10	0x10(AM10)	0x22	0x30(PM10)
0x11	0x11(AM11)	0x23	0x31(PM11)

=AM time

=PM time+0x20

28

RTC Example

- RTC sample code with driver
  - Enable RTC and set date and time
    - Date: 2009.1.19
    - Time:13.20.00
  - Enable alarm interrupt and set alarm date and time
    - Alarm date:2009.1.19
    - Alarm Time:13.20.10
- RTC sample code
  - The same flow

29

Smpl_RTC



30

Smpl_RTC RTC register access


```
#include <stdio.h>
#include <string.h>
#include "NUC1xx.h"
#include "LCD.h"

static uint8_t Alarm_E=1;

// RTC register access
void set_TLR (int32_t a,int32_t b,int32_t c,int32_t d,int32_t e,int32_t f)
{
      outpw(&RTC->TLR, a<<20|b<<16|c<<12|d<<8|e<<4|f)                ;
}
void set_CLR (int32_t a,int32_t b,int32_t c,int32_t d,int32_t e,int32_t f)
{
      outpw(&RTC->CLR, a<<20|b<<16|c<<12|d<<8|e<<4|f)                ;
}
void set_TAR(int32_t a,int32_t b,int32_t c,int32_t d,int32_t e,int32_t f)
{
      outpw(&RTC->TAR, a<<20|b<<16|c<<12|d<<8|e<<4|f)                ;
}
void set_CAR (int32_t a,int32_t b,int32_t c,int32_t d,int32_t e,int32_t f)
{
      outpw(&RTC->CAR, a<<20|b<<16|c<<12|d<<8|e<<4|f)                ;
}
```

31

START_RTC()


```
// start Real-Time Clock
void START_RTC(void)
{
    while(1)
    {
        RTC->INIR = 0xa5eb1357;
        if(inpw(&RTC->INIR)==1)
        break;
    }
    while(1)
    {
        RTC->AER.AER = 0xA965;
    if(inpw(&RTC->AER)&0x10000)// AER bit
        break;
  }
}
```


InitRTC()


```
void InitRTC(void)
{
    UNLOCKREG();
    /* Step 1. Enable and Select RTC clock source */
    SYSCLK->PWRCON.XTL32K_EN = 1;//Enable 32Khz for RTC clock source
    SYSCLK->APBCLK.RTC_EN =1;//Enable RTC clock source
    /* Step 2. Initiate and unlock RTC module */
    START_RTC();
    /* Step 3. Initiate Time and Calendar  setting */
    RTC->TSSR.HR24_HR12     =1;//Set 24hour mode
    //Set time and calendar for loading register, Calendar 09/1/19, Time 13:20:00
    set_CLR(0,9,0,1,1,9);
    set_TLR(1,3,2,0,0,0);
    /* Step 4. Set alarm interrupt */
    //Set time and calendar for alarm register , Calendar 09/1/19, Time 13:30:00
    set_CAR(0,9,0,1,1,9);
    set_TAR(1,3,2,0,2,0);
    //Enable alarm interrupt
    RTC->RIER.AIER = 1;
    RTC->RIER.TIER = 1;
    NVIC_EnableIRQ(RTC_IRQn);
}
```

Smpl_RTC main()


```
int32_t main (void)
{
    UNLOCKREG();
    SYSCLK->PWRCON.XTL32K_EN = 1;//Enable 32KHz for RTC clock source
    SYSCLK->PWRCON.XTL12M_EN = 1;//Enable 12MHz crystal
    SYSCLK->CLKSEL0.HCLK_S = 0;
    LOCKREG();
    init_LCD();
    clear_LCD();
    print_Line(0,"Smpl_RTC");
    InitRTC();
    /* Synch field transmission & Request Identifier Field transmission*/
    while(Alarm_E)
    {
        UNLOCKREG();
        WDT->WTCR.WTR =1;//Write 1 to clear for safety
        LOCKREG();
    }
    while(1)
    {
        __NOP();
    }
}
```


Smpl_Timer_WDT_RTC



Smpl_Timer_WDT_RTC


```
int32_t main (void)
{
    UNLOCKREG();
    SYSCLK->PWRCON.XTL32K_EN = 1;//Enable 32KHz for RTC clock source
    SYSCLK->PWRCON.XTL12M_EN = 1;//Enable 12MHz crystal
    SYSCLK->CLKSEL0.HCLK_S = 0;
    LOCKREG();
    Initial_panel();
    clr_all_panel();

    InitTIMER0();
    InitRTC();
    InitWDT();

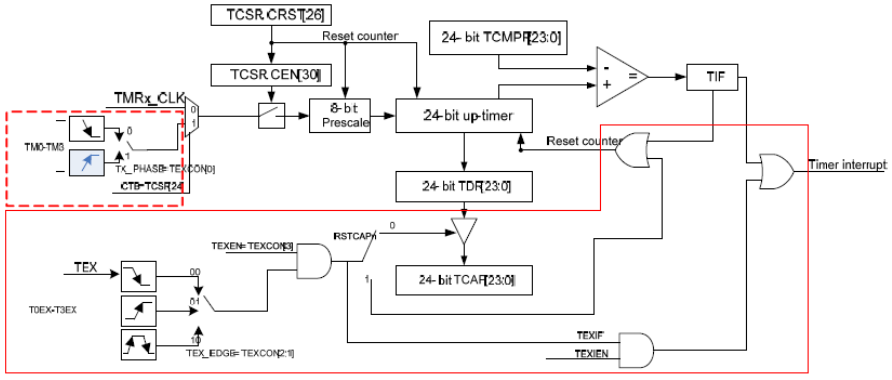
    /* Synch field transmission & Request Identifier Field transmission*/
    while(Alarm_E)
    {
        UNLOCKREG();
        WDT->WTCR.WTR =1;//Write 1 to clear for safety
        LOCKREG();
    }
    while(1)
    {
        __NOP();
    }
}
```


Timer Capture pins

Based on NUC140 LQFP 100pin

- T0EX : GPB15 /INT1 pin91 (Timer0 Capture input pin)
- T1EX : GPE5 / PWM5 pin47 (Timer1 Capture input pin)
- T2EX : GPB2 / RTS0 pin34 (Timer2 Capture input pin)
- T3EX : GPB3 / CTS0 pin35 (Timer3 Capture input pin)

Timer Block Diagram



Timer Capture Inputs

- If TEXEN (Timer External Pin Enable) is set to 1 and RSTCAPSEL is set to 0, the timer counter value (TDR) will be captured into TCAP register when TEX (Timer External Pin) pin trigger condition occurred.
- If TEXEN is set to 1 and RSTCAPSEL is set to 1, the TDR will be reset to 0 when TEX pin trigger condition happened.
- The TEX trigger edge can choose by TEX_EDGE.
- When TEX trigger occurred, TEXIF (Timer External Interrupt Flag) is set to 1, and if enabled TEXIEN (Timer External Interrupt Enable Bit) to 1, the interrupt signal is generated then sent to NVIC to inform CPU.
- It also provides T0EX~T3EX enabled or disabled capture de-bounce function by TEXCONx[6]. And, the TEX source operating frequency should be less than 1/3 HCLK frequency if disable TEX de-bounce or less than 1/8 HCLK frequency if enable TEX de-bounce.